

URL Shortener — Practical System Design Notes

Hinglish • Base62 • Redis • Caching • Analytics • Scaling • Interview-focused

1. URL Shortener Kya Hai?

URL shortener long URL ko ek small unique URL mein convert karta hai. Example:
`https://example.com/products/category/item/123` → `https://sho.rt/X7kP2`.

```
Long URL
  ↓
POST /shorten
  ↓
Generate short ID
  ↓
Store mapping
  ↓
Short URL

GET /X7kP2
  ↓
Find original URL
  ↓
302 Redirect
```

2. Main Problem

Short URL banana easy hai. Real system-design challenge hai: unique ID generate karna, billions of redirects low latency par serve karna, hot URLs handle karna, expiration/analytics support karna aur abuse control karna.

3. Functional Requirements

- Long URL ko short URL mein convert karna.
- Short URL ko original URL par redirect karna.
- Optional custom alias.
- Optional expiration.
- Click/analytics tracking.
- User-specific URL management, if required.

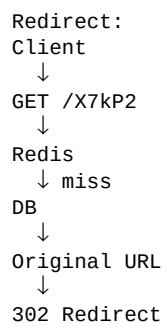
4. Non-Functional Requirements

- Low redirect latency.
- High read/redirect availability.
- Unique short IDs.
- Horizontal scalability.
- Reliable storage.
- Rate limiting and abuse protection.

5. Basic Architecture

```
Client
  ↓
Load Balancer
  ↓
URL API × N
  ■■■ Redis Cache
```

Database



6. DB Schema

URL

```
-----  
id  
short_code  
original_url  
user_id  
created_at  
expires_at  
status
```

Unique:
short_code

short_code par unique index critical hai. original_url ko unnecessarily index karne ki zaroorat nahi jab primary lookup short_code se ho.

7. Short ID Generation — Option 1: Random

Generate random:
X7kP2a

Check DB:
exists?
YES → generate again
NO → save

Random generation simple hai, but collision probability ko account karna padta hai. Scale badhne par retries/entropy carefully design karo.

8. Short ID Generation — Option 2: Hash

```
Long URL  
↓  
hash  
↓  
abcdef123456...
```

Take first N chars
↓
short code

Hash deterministic ho sakta hai, lekin truncation collision create kar sakti hai. Isliye collision handling still required hai.

9. Base62 — Important

Base62 mein 62 characters use hote hain: a-z, A-Z, 0-9. Numeric ID ko compact alphanumeric string mein encode kiya ja sakta hai.

Characters:
26 lowercase
26 uppercase
10 digits
= 62

Example:
DB ID = 125000
↓
Base62
↓
X7kP2

10. Counter + Base62

DB sequence:
100001
100002
100003

↓
Base62
↓
q8x
q8y
q8z

Monotonic numeric IDs + Base62 deterministic short codes de sakte hain. Distributed ID generation ko multiple writers ke saath carefully handle karna padta hai.

11. Base62 Length

Approximate capacity: length 1 = 62 values, length 2 = 62^2 , length 3 = 62^3 , etc. Isliye code length ko expected URL volume ke according choose karo.

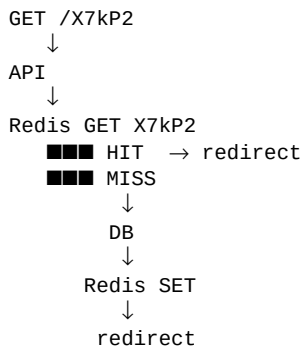
$62^1 = 62$
 $62^2 = 3,844$
 $62^3 = 238,328$
 $62^4 = 14,776,336$
 $62^5 = 916,132,832$
 $62^6 = 56,800,235,584$

12. Collision

Collision ka matlab do different long URLs ko same short_code mil jana. Unique DB constraint final safety net hona chahiye.

Generate short_code
↓
INSERT
↓
Unique constraint
 ■■■ success → done
 ■■■ conflict → generate another / retry

13. Redirect Flow



Redirect path generally read-heavy hota hai, isliye caching extremely useful ho sakti hai.

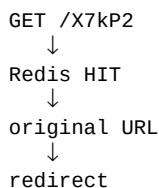
14. 301 vs 302

302	Temporary redirect; client/proxy caching behavior generally more controllable.
301	Permanent redirect; aggressive caching can make future behavior harder to change.

Choice product requirements, SEO behavior and caching/control requirements par depend karta hai.

15. Redis Cache

Redis:
short_code → original_url

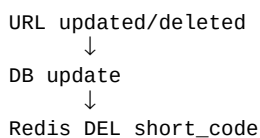


Popular URLs ke liye cache DB load dramatically reduce kar sakta hai.

16. Cache TTL

URL expiry aur cache TTL different concepts hain. A URL valid for 30 days ho sakta hai but Redis entry ka TTL shorter ho sakta hai; cache miss par DB se reload kiya ja sakta hai.

17. Cache Invalidation

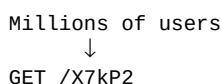


Agar URL destination editable hai, stale cache avoid karna important hai.

18. Cache Stampede

Agar hot key expire hote hi thousands requests DB par hit kar dein, DB spike ho sakta hai. TTL jitter, request coalescing/single-flight, background refresh ya other cache strategies use ki ja sakti hain.

19. Hot URL



↓
same Redis key
↓
hot key

Hot URLs ke liye cache placement, replication, CDN/edge caching and request distribution carefully design kiye ja sakte hain.

20. Custom Alias

```
POST /shorten
{
  "url": "...",
  "alias": "sonu"
}
```

Result:
sho.rt/sonu

Alias globally unique hona chahiye. Reserved words jaise admin, login, api etc. block kiye ja sakte hain.

21. URL Expiration

created_at
expires_at

```
GET /X7kP2
↓
expired?
  ■■■ YES → 404 / expired response
  ■■■ NO → redirect
```

Expired URLs ko cleanup/archive karne ke liye background job ya database TTL/lifecycle capability use ki ja sakti hai, depending on storage.

22. Click Analytics

```
GET /X7kP2
↓
redirect
↓
analytics event
↓
Queue
↓
Analytics Worker
↓
Analytics DB
```

Redirect request ke critical path mein heavy analytics DB write karna latency aur availability ko hurt kar sakta hai. Event ko queue karke async processing useful hai.

23. Analytics Data

```
ClickEvent
-----
short_code
timestamp
country
device
browser
referrer
ip_hash / privacy-safe identifier
user_agent
```

Exact fields privacy, legal requirements aur product needs ke according choose karo.

24. Analytics Aggregation

```
Raw events
↓
Hourly aggregation
↓
Daily aggregation
↓
Dashboard
```

clicks/day
clicks/country
device split
referrer

25. Rate Limiting

Shorten endpoint ko abuse se protect karna important hai. Redirect endpoint ko bhi abusive traffic, scraping aur bot traffic ke against appropriate controls ki zaroorat ho sakti hai.

POST /shorten
↓
Rate Limiter
↓
API

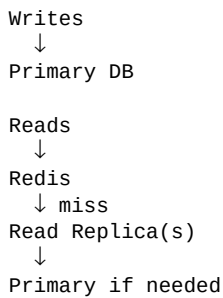
Examples:
per IP
per user
per API key
per tenant

26. Abuse / Malicious URLs

- Malware/phishing URL detection.
- Blocklists or reputation checks.
- Rate limiting.
- Report/disable mechanism.
- Reserved aliases.
- Authentication for management APIs.

27. Database Scaling

URL shortener ka workload usually read-heavy hota hai: writes comparatively low, redirects potentially very high. Isliye cache + read scaling important.



28. Read Replicas

Read replicas redirect lookup traffic distribute kar sakte hain, but replication lag consider karo. Redis cache hit rate high ho to database traffic aur reduce hota hai.

29. Sharding

Extreme scale par URL mappings ko multiple database shards mein distribute kiya ja sakta hai. Short-code hash/range based shard routing possible hai, but operational complexity increase hoti hai.

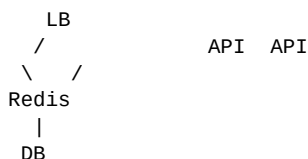
30. Partitioning vs Sharding

Partitioning	One logical DB/table ko partitions mein split karna.
Sharding	Data ko multiple independent DB nodes/shards mein distribute karna.

31. CDN / Edge Caching

Redirect responses kuch architectures mein edge/CDN par cache kiye ja sakte hain, especially stable mappings. But cache-control and invalidation semantics carefully choose karo.

32. Availability



Redis failure:
→ fallback to DB
→ avoid total outage if acceptable

Fail-open/fail-closed behavior depends on whether the component is cache, security control, or source of truth.

33. Idempotent Create

Shorten API ko idempotent banana useful ho sakta hai, especially when the client retries. A client-provided idempotency key can prevent accidental creation of multiple short URLs for one logical operation.

34. API Design

```
POST /api/urls
{
  "url": "https://example.com/long/path"
}
```

```
Response:
{
  "shortCode": "X7kP2",
  "shortUrl": "https://sho.rt/X7kP2"
}
```

```
GET /X7kP2
→ 302 Location: original URL
```

```
DELETE /api/urls/X7kP2
→ disable/delete according to policy
```

35. Security

- Validate URL format and allowed schemes.
- Do not allow dangerous protocols such as javascript: in normal web-link creation.
- Rate-limit creation.
- Protect admin/management APIs.
- Prevent open redirect abuse according to product policy.
- Log security events.
- Use authentication where user ownership is required.

36. Open Redirect Consideration

A URL shortener intentionally redirects users, so destination control is central to the product. For enterprise/internal systems, allowlists may be appropriate. For public systems, abuse detection and takedown mechanisms become important.

37. Local Node.js Implementation

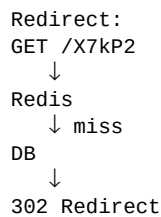
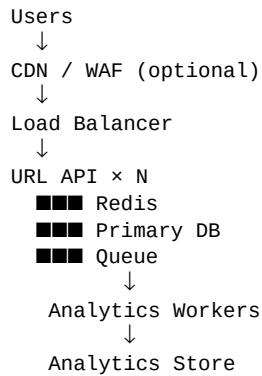
```
POST /shorten
↓
validate URL
↓
generate ID
↓
DB insert
↓
Redis set
↓
return short URL
```

```
GET /:code
↓
Redis get
↓ miss
DB query
↓
Redis set
↓
302 redirect
```

38. Suggested Stack

Node.js + Express/Fastify + Redis + PostgreSQL/MySQL/MongoDB + BullMQ for async analytics. Start with one API server and one DB; then add Redis and multiple API instances.

39. Production Architecture



40. End-to-End Shorten Flow

1. Client sends long URL
2. API authenticates/rate-limits if needed
3. Validate URL
4. Generate short code
5. Insert mapping with unique constraint
6. Cache mapping in Redis
7. Return short URL

41. End-to-End Redirect Flow

1. GET /X7kP2
2. Check Redis
3. Cache HIT → redirect
4. Cache MISS → DB
5. Validate expiry/status
6. Populate cache
7. Return redirect
8. Publish analytics asynchronously

42. Failure Scenarios

Failure	Handling
Redis down	Fallback to DB if latency/availability trade-off is acceptable
DB unavailable	Existing cached redirects may continue; creation may fail
Collision	Unique constraint + retry
Analytics queue down	Redirect should not necessarily fail; buffer/retry according to design
Hot key	Caching/edge strategies and distributed load
Malicious URL	Validation, reputation checks, disable/report controls
Client retry	Idempotency where create semantics require it

43. Observability / Grafana

- Shorten requests/sec
- Redirect requests/sec
- Redirect p50/p95/p99 latency
- Redis hit ratio
- Redis latency/errors
- DB query latency
- DB CPU/connections
- Cache misses
- Collision/retry count
- Analytics queue depth
- Worker processing latency
- 404/expired URL rate
- Abuse/rate-limit responses

44. Interview Q&A;

Question	Short answer
Short ID kaise generate?	Random, hash-based, or numeric ID + Base62; trade-offs explain karo.
Base62 kya hai?	62-character alphabet se number ko compact string mein encode karna.
Collision kaise handle?	Unique DB constraint + retry; generation strategy carefully choose karo.
Redis kyun?	Redirect lookup read-heavy hai; cache latency aur DB load reduce karta hai.
301 vs 302?	Permanent vs temporary redirect semantics; caching behavior differs.
Analytics synchronous kyun nahi?	Redirect latency aur critical path ko heavy writes se isolate karna.
Hot URL?	High-frequency same key; caching/edge strategies help.
Read replicas?	DB reads distribute karte hain, but lag consider karo.
Custom alias?	User-defined short code with uniqueness/reserved-word checks.
Expiration?	expires_at/state check; cleanup asynchronously.
Sharding?	Extreme scale par mappings ko multiple DB nodes mein distribute karna.
Security?	URL validation, abuse detection, rate limiting, auth and takedown controls.

45. Practice Scenarios

1. Design a URL shortener for 100 million URLs.
2. Redirect traffic is 100x creation traffic. How will you design the system?
3. Explain Base62 with a numeric ID.
4. How do you handle short-code collision?
5. Redis is down during a traffic spike. What happens?
6. One short URL receives 1 million requests/minute. How do you protect the DB?
7. Add URL expiration.

8. Add custom aliases.
9. Add click analytics without increasing redirect latency.
10. Design analytics for 1 billion clicks/day.
11. What happens when a DB read replica is lagging?
12. How would you prevent your public URL shortener from becoming a phishing platform?

46. Local Practice Project

Node.js + Express + Redis + MySQL/PostgreSQL se URL shortener banao. First simple version: POST /shorten + GET /:code. Then numeric ID + Base62, unique constraint, Redis cache, TTL/expiration, custom alias, rate limiting, BullMQ analytics and Prometheus/Grafana metrics add karo.

Phase 1:

POST /shorten
GET /:code

Phase 2:

DB + unique short_code

Phase 3:

Base62

Phase 4:

Redis cache

Phase 5:

Expiration + custom alias

Phase 6:

BullMQ analytics

Phase 7:

Rate limiting

Phase 8:

Prometheus + Grafana

Phase 9:

Run 3 Node instances
behind a load balancer

47. Final Cheat Sheet

URL SHORTENER

CREATE:

Client

↓

API

↓

Validate

↓

Generate Short ID

↓

DB

↓

Redis

↓

Short URL

REDIRECT:

GET /code

↓

Redis

↓ HIT → 302

↓ MISS

DB

↓

Redis

↓

302

Important:

→ Base62

→ collision handling

→ unique constraint

→ Redis caching

→ hot keys

- 301 vs 302
- expiration
- custom aliases
- analytics queue
- rate limiting
- abuse prevention
- DB scaling
- sharding

Golden interview line:

"I'll keep creation and redirect paths separate, use a collision-safe short-code strategy with a unique DB constraint, cache hot mappings in Redis, keep analytics asynchronous, and scale the read-heavy redirect path independently."